

Machine Learning(unit-1)

Q What is Machine Learning?

Machine Learning (ML) is a branch of artificial intelligence (AI) that allows computers to learn from data and improve their performance over time without being explicitly programmed. Instead of following hard-coded instructions, machine learning algorithms identify patterns and make decisions based on input data.

Key Terminologies in Machine Learning

Data: The information you give to the computer to learn from. For example, if you're teaching it to recognize fruits, the data could be pictures of apples, bananas, etc.

Feature: A characteristic or property of your data. In the fruit example, features could be color, size, and shape of the fruits.

Label: The correct answer or outcome you want the machine to predict. If you're teaching the computer to recognize fruits, the label would be "apple," "banana," etc.

Model: The system or program that learns from the data to make predictions. It's like a recipe that's been trained to recognize patterns.

Training: The process of teaching the model using data. The model looks at the data and learns patterns from it.

Testing: After the model is trained, you give it new data to see how well it performs.

Overfitting: When the model learns too much detail from the training data, including noise or irrelevant patterns, and doesn't work well on new data.

Underfitting: When the model is too simple and doesn't learn enough from the data, so it performs poorly both on training and new data.

Accuracy: A measure of how often the model makes the correct prediction. If the model predicts correctly 90 out of 100 times, its accuracy is 90%.

Loss Function: A function that tells the model how wrong its predictions are. The goal is to make this number as small as possible.

Gradient Descent: A method used by the model to adjust itself and reduce errors. It's like a hill-climbing strategy to reach the best solution.

Types of Machine Learning

ANS- 1. Supervised Learning

- **How it works:** The model is trained on labeled data, which means the data comes with correct answers (called labels). The model learns to predict the labels for new data.

- **Example:** Teaching a model to recognize cats and dogs. You provide it with labeled images ("cat" or "dog") and it learns to classify new images.

Categories of supervised machine learning:

1. **Classification**
2. **Regression**

Classification

Classification algorithms are used to solve the classification problems in which the output variable is categorical, such as "Yes" or No, Male or Female, Red or Blue, etc. The classification algorithms predict the categories present in the dataset. Some real-world examples of classification algorithms are Spam Detection, Email filtering, etc.

Some popular classification algorithms are given below:

- **Random Forest Algorithm**
- **Decision Tree Algorithm**
- **Logistic Regression Algorithm**
- **Support Vector Machine Algorithm**

Regression

Regression algorithms are used to solve regression problems in which there is a linear relationship between input and output variables. These are used to predict continuous output variables, such as market trends, weather prediction, etc.

Some popular Regression algorithms are given below:

- **Simple Linear Regression Algorithm**
- **Multivariate Regression Algorithm**
- **Decision Tree Algorithm**
- **Lasso Regression**

2. Unsupervised Learning

- **How it works:** The model is trained on data that isn't labeled, so it tries to find hidden patterns or groupings in the data.
- **Example:** Grouping customers based on their shopping behavior without knowing their categories beforehand.
- **Common Algorithms:** Clustering (like K-means), principal component analysis (PCA), and association rules.

3. Semi-Supervised Learning:

- Semi-supervised learning is a mix of supervised and unsupervised learning. The model is trained on a small amount of labeled data and a large amount of unlabeled data. This approach is useful when labeling data is expensive or time-consuming.
- **Example:** In image recognition, only a few images are labeled (e.g., “cat” or “dog”), while most of the images are unlabeled. The model uses the labeled images to guide its learning and improves its ability to label the unlabeled images.

Common Algorithms:

- **Self-training**
- **Co-training**
- **Generative models (e.g., Variational Autoencoders)**

4. Reinforcement Learning:

- In reinforcement learning, the model learns by interacting with its environment and receiving feedback in the form of rewards or penalties. It tries to take actions that maximize the cumulative reward over time.
- **Example:** A robot learning to walk by trying different movements and getting rewarded for successful steps.

Common Algorithms:

- **Q-learning**
- **Deep Q Networks (DQN)**
- **Policy Gradients**
- **Actor-Critic Methods**

Perspectives and issues in machine learning

ANS-Perspectives in Machine Learning:

1. Data-Centric vs. Model-Centric:

- **Data-Centric:** Emphasizes the quality of data. This perspective focuses on improving data collection, cleaning, and representation. Better data leads to better models.
- **Model-Centric:** Focuses on designing and improving algorithms. Here, the belief is that more complex models or optimizations yield better results, even with noisy or incomplete data.

2. Interpretability vs. Performance:

- **Interpretability:** Simple, interpretable models like linear regression or decision trees are often preferred in domains where understanding the model's decisions is critical (e.g., healthcare).
 - **Performance:** Complex models (e.g., deep learning) often yield better results, but their decision processes are harder to explain.
3. **Supervised vs. Unsupervised vs. Reinforcement Learning:**
- **Supervised Learning:** Learning from labeled data (e.g., classification, regression).
 - **Unsupervised Learning:** Identifying patterns or clusters in data without labels (e.g., clustering).
 - **Reinforcement Learning:** Learning from interaction with an environment (e.g., game playing).
4. **Theoretical vs. Applied:**
- **Theoretical:** Focus on developing new algorithms, understanding their mathematical properties, and ensuring robustness.
 - **Applied:** Focus on solving real-world problems, often tweaking or applying existing algorithms.

Key Issues in Machine Learning:

1. **Data Quality:** Machine learning models heavily depend on good data. Inaccurate, biased, or incomplete data can lead to poor model performance and wrong predictions.
2. **Overfitting and Underfitting:**
 - **Overfitting:** When a model is too complex, it learns noise in the training data and performs poorly on unseen data.
 - **Underfitting:** When a model is too simple, it cannot capture the underlying patterns in the data, leading to poor performance.
3. **Bias and Fairness:**
 - Models can inherit biases from the data they're trained on, leading to unfair or discriminatory predictions, especially in sensitive domains like hiring or lending.
4. **Model Interpretability:** As models grow more complex (e.g., deep learning), understanding why they make certain predictions becomes difficult. This is a concern in domains where transparency is important.
5. **Scalability:** Some algorithms don't scale well with large datasets, requiring significant computational resources (memory, processing power).

6. **Privacy and Security:**

- Using large datasets, especially with sensitive information, raises concerns about data privacy and security breaches.

7. **Transfer Learning:** Models trained on one task or dataset might not generalize well to another task or dataset, a common challenge when data is limited.

8. **Ethics in AI:** Concerns about job displacement, decisions made by machines, and the ethical use of AI in decision-making.

Applications of machine learning

ANS-Machine Learning (ML) has a wide range of applications across various fields. Here are some key applications:

1. Healthcare

- **Disease Diagnosis:** ML helps doctors spot diseases from medical images or patient data.
- **Drug Discovery:** ML speeds up finding new medicines by predicting how they'll work.
- **Personalized Medicine:** ML suggests treatments based on individual patient data.

2. Finance

- **Fraud Detection:** ML spots unusual transactions to prevent credit card fraud.
- **Algorithmic Trading:** ML predicts stock prices and helps automate buying and selling.
- **Risk Assessment:** ML helps banks decide who can get loans by analyzing customer information.

3. Retail and E-commerce

- **Recommendation Systems:** ML suggests products or movies based on what you like.
- **Customer Segmentation:** ML groups customers by their shopping habits to target ads better.
- **Price Optimization:** ML helps set the best prices based on market trends.

4. Manufacturing

- **Predictive Maintenance:** ML predicts when machines might break down, so they can be fixed before they fail.
- **Quality Control:** ML checks products for defects during production to ensure quality.

5. Autonomous Vehicles

- **Self-driving Cars:** ML helps self-driving cars understand their surroundings and make driving decisions.
- **Traffic Management:** ML predicts traffic jams and suggests better routes.

6. Natural Language Processing (NLP)

- **Speech Recognition:** ML converts spoken words into text, used in voice assistants like Siri.
- **Language Translation:** ML translates text between different languages.
- **Chatbots:** ML powers chatbots that can answer questions and assist customers.

7. Image and Video Processing

- **Facial Recognition:** ML identifies people by their faces, used in security and social media.
- **Object Detection:** ML spots and labels objects in photos and videos.
- **Content Generation:** ML can create new images or videos, like deepfakes or AI-generated art.

8. Agriculture

- **Crop Monitoring:** ML checks crop health using images and sensors to improve farming.
- **Yield Prediction:** ML predicts how much crop will be harvested based on various factors.
- **Robotics and Automation:** ML-powered robots help with planting and harvesting crops.

9. Cybersecurity

- **Anomaly Detection:** ML finds unusual activities in computer systems to prevent cyberattacks.
- **Threat Detection:** ML helps identify new types of malware and attacks.

10. Entertainment

- **Content Recommendation:** ML suggests songs or videos you might like based on your past choices.
- **Game AI:** ML creates smart behavior for characters in video games.
- **Content Creation:** ML helps generate new music, art, or stories.

11. Education

- **Personalized Learning:** ML customizes lessons based on a student's strengths and weaknesses.

- **Automated Grading:** ML can grade assignments automatically, especially for subjects like math.

12. Energy

- **Energy Consumption Prediction:** ML predicts how much energy will be used and optimizes its distribution.
- **Smart Grids:** ML manages electricity distribution efficiently and integrates renewable energy sources.

Datasets and its types

ANS- In machine learning, datasets are collections of data used for training, validating, and testing models. Different types of datasets are suited for various tasks and learning methods. Here's an overview of common dataset types and their characteristics:

**Types of Datasets

1.1. Structured Datasets

- **Definition:** Data organized into rows and columns, typically in tabular form (e.g., spreadsheets, SQL databases).
- **Example:** A dataset containing information about customers with columns for age, income, and purchase history.
- **Usage:** Used in traditional machine learning tasks like regression, classification, and clustering.

1.2. Unstructured Datasets

- **Definition:** Data that doesn't fit into a tabular format and lacks a predefined structure (e.g., text, images, audio).
- **Example:** A collection of customer reviews (text data) or a set of photographs (image data).
- **Usage:** Used in natural language processing (NLP), computer vision, and audio analysis.

** Dataset Categories

**2.1. Training Dataset

- **Definition:** The portion of the dataset used to train the model. It's where the model learns the patterns and relationships in the data.
- **Characteristics:** Contains a large amount of data to help the model learn effectively.
- **Example:** A dataset with thousands of labeled images used to train an image classifier.

****2.2. Validation Dataset**

- **Definition:** A subset of the dataset used to tune hyperparameters and make decisions about the model's architecture. It helps in evaluating the model's performance during training.
- **Characteristics:** It's separate from the training data and used to check how well the model generalizes to unseen data.
- **Example:** Data used to adjust the learning rate or choose between different models.

****2.3. Test Dataset**

- **Definition:** A separate subset of the dataset used to assess the final performance of the model after training and validation. It provides an unbiased evaluation of the model's accuracy.
- **Characteristics:** It's not used during training or validation and is used to estimate how the model will perform on real-world data.
- **Example:** Data used to evaluate the accuracy of a classifier after it has been trained and validated.

**** Types of Data**

****3.1. Categorical Data**

- **Definition:** Data that represents categories or groups. It can be nominal (no inherent order) or ordinal (with a meaningful order).
- **Example:** Colors (nominal), education levels (ordinal).
- **Usage:** Often encoded into numerical values for machine learning models (e.g., one-hot encoding).

****3.2. Numerical Data**

- **Definition:** Data that represents measurable quantities and is expressed in numerical form.
- **Example:** Height, weight, temperature.
- **Types:**
 - **Discrete Data:** Countable values (e.g., number of children).
 - **Continuous Data:** Measurable and can take any value within a range (e.g., height, weight).

****3.3. Text Data**

- **Definition:** Data consisting of textual information.
- **Example:** Reviews, articles, tweets.

- **Usage:** Processed using techniques from natural language processing (NLP) like tokenization and embedding.

****3.4. Image Data**

- **Definition:** Data consisting of visual information.
- **Example:** Photographs, medical imaging.
- **Usage:** Processed using techniques from computer vision like convolutional neural networks (CNNs).

****3.5. Time-Series Data**

- **Definition:** Data collected sequentially over time, often used to observe trends and patterns.
- **Example:** Stock prices, weather data.
- **Usage:** Analyzed using time-series analysis techniques and models like Long Short-Term Memory (LSTM) networks.

****3.6. Audio Data**

- **Definition:** Data consisting of sound recordings or audio signals.
- **Example:** Speech recordings, music tracks.
- **Usage:** Processed using techniques from audio signal processing and machine learning models for speech recognition or audio classification.

****4. Dataset Sources**

- **Public Datasets:** Available online for various domains (e.g., ImageNet for images, UCI Machine Learning Repository for tabular data).
- **Synthetic Datasets:** Generated artificially, often used when real data is scarce or to simulate specific conditions.
- **Real-World Datasets:** Collected from actual applications or user interactions, often requiring careful handling of privacy and ethics.

Data Preprocessing in Machine learning

Data preprocessing is a process of preparing the raw data and making it suitable for a machine learning model. It is the first and crucial step while creating a machine learning model.

When creating a machine learning project, it is not always a case that we come across the clean and formatted data. And while doing any operation with data, it is mandatory to clean it and put in a formatted way. So for this, we use data preprocessing task.

Why do we need Data Preprocessing?

A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models. Data preprocessing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.

It involves below steps:

- **Getting the dataset**
- **Importing libraries**
- **Importing datasets**
- **Finding Missing Data**
- **Encoding Categorical Data**
- **Splitting dataset into training and test set**
- **Feature scaling**

1) Get the Dataset

To create a machine learning model, the first thing we required is a dataset as a machine learning model completely works on data. The collected data for a particular problem in a proper format is known as the **dataset**.

2) Importing Libraries

In order to perform data preprocessing using Python, we need to import some predefined Python libraries. These libraries are used to perform some specific jobs. There are three specific libraries that we will use for data preprocessing, which are:

Numpy- used for mathematical calculations.

Matplotlib- used to plot any types of charts,

Pandas- used for importing and managing the dataset.

3) Importing the Datasets

Now we need to import the datasets which we have collected for our machine learning project. But before importing a dataset, we need to set the current directory as a working directory.

4) Handling Missing data:

The next step of data preprocessing is to handle missing data in the datasets. If our dataset contains some missing data, then it may create a huge problem for our machine learning model. Hence it is necessary to handle missing values present in the dataset.

Ways to handle missing data:

There are mainly two ways to handle missing data, which are:

By deleting the particular row: The first way is used to commonly deal with null values. In this way, we just delete the specific row or column which consists of null values. But this way is not so efficient and removing data may lead to loss of information which will not give the accurate output.

By calculating the mean: In this way, we will calculate the mean of that column or row which contains any missing value and will put it on the place of missing value.

5) Encoding Categorical data:

Categorical data is data which has some categories such as colors, education level etc.

Since machine learning model completely works on mathematics and numbers, but if our dataset would have a categorical variable, then it may create trouble while building the model. So it is necessary to encode these categorical variables into numbers.

6) Splitting the Dataset into the Training set and Test set

In machine learning data preprocessing, we divide our dataset into a training set and test set. This is one of the crucial steps of data preprocessing as by doing this, we can enhance the performance of our machine learning model.

Training Set: A subset of dataset to train the machine learning model, and we already know the output.

Test set: A subset of dataset to test the machine learning model, and by using the test set, model predicts the output.

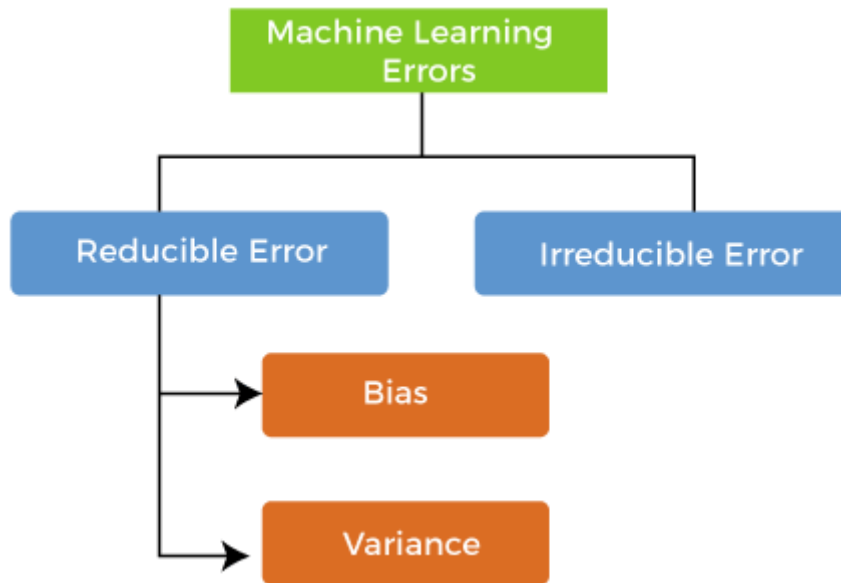
7) Feature Scaling

Feature scaling is the final step of data preprocessing in machine learning. It is a technique to standardize the independent variables of the dataset in a specific range. In feature scaling, we put our variables in the same range and in the same scale so that no any variable dominate the other variable.

Bias and Variance in Machine Learning

ANS- Machine learning is a branch of Artificial Intelligence, which allows machines to perform data analysis and make predictions. However, if the machine learning model is not accurate, it can make predictions errors, and these prediction errors are usually known as Bias and Variance. In machine learning, these errors will always be present as there is always a slight difference between the model predictions and actual predictions.

ERROR- It is the difference between the prediction made by a model and the actual value or result.



Reducible errors: These errors can be reduced to improve the model accuracy. Such errors can further be classified into bias and Variance.

Irreducible errors: These errors will always be present in the model regardless of which algorithm has been used. The cause of these errors is unknown variables whose value can't be reduced.

What is Bias?

While making predictions, a difference occurs between prediction values made by the model and actual values/expected values, and this difference is known as bias errors or Errors due to bias.

Low bias means the model is good at learning the patterns in the data.

High bias means the model is too simple and doesn't learn enough from the data.

Ways to reduce High Bias:

1. Use a more complex model.
2. Add more features.
3. Decrease the regularization term.

What is a Variance Error?

The variance would specify the amount of variation in the prediction if the different training data was used. In simple words, **variance tells that how much a random variable is different from its expected value.**

Low Variance- It means the model gives almost the same results every time, even if you change the training data a little.

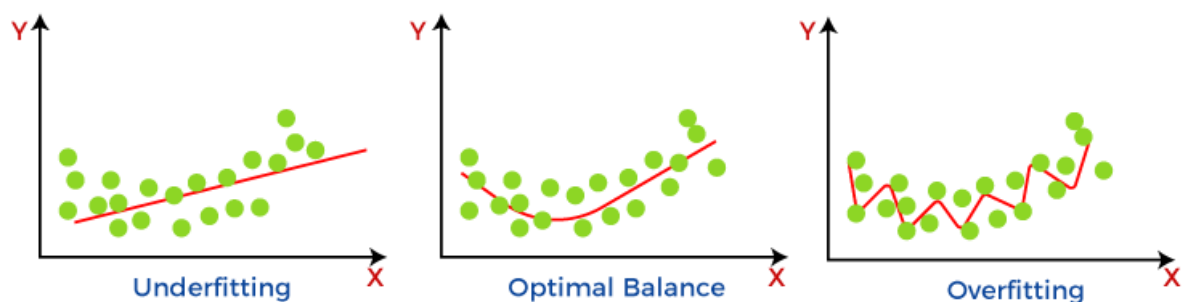
High Variance- It means the model changes a lot when you give it different data. It's too focused on the details of the training data.

High variance has the below problems:

- A high variance model leads to overfitting.
- Increase model complexities.

Low variance has the below problems:

- A low variance model leads to underfitting.
- Limited Improvement.

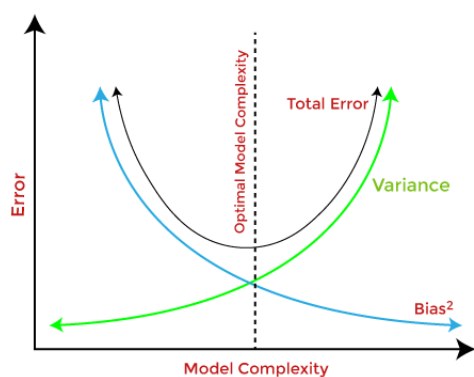


Ways to Reduce High Variance:

- Reduce the number of features.
- Use a simpler model.
- Increase the training data.
- Increase the Regularization term.

Bias-Variance Trade-Off

While building the machine learning model, it is really important to take care of bias and variance in order to avoid overfitting and underfitting in the model. If the model is very simple with fewer parameters, it may have low variance and high bias. Whereas, if the model has a large number of parameters, it will have high variance and low bias. So, it is required to make a balance between bias and variance errors, and this balance between the bias error and variance error is known as **the Bias-Variance trade-off**.



For an accurate prediction of the model, algorithms need a low variance and low bias. But this is not possible because bias and variance are related to each other:

- If we decrease the variance, it will increase the bias.
- If we decrease the bias, it will increase the variance.

Hence, the **Bias-Variance trade-off is about finding the sweet spot to make a balance between bias and variance errors.**

Underfitting

- **Definition:** Underfitting happens when a model is too simple to capture the underlying patterns in the data. It doesn't learn enough from the training data, leading to poor performance on both training and new data.
- **Example:** Imagine trying to predict house prices using only one feature, like the size of the house, when in reality, factors like location and condition also matter. The model might make general, inaccurate predictions because it's not considering all relevant information.

Overfitting

- **Definition:** Overfitting occurs when a model is too complex and learns not only the useful patterns but also the noise or irrelevant details from the training data. This makes it perform very well on the training data but poorly on new, unseen data.

Real-World Overfitting Example

Scenario: Predicting student test scores based on their study hours and other factors.

1. **Problem:** Imagine a teacher wants to predict students' test scores based on how many hours they study, their previous grades, and additional details like their favorite study snacks or the color of their notebooks.
2. **Model:** The teacher uses a very complex model that takes into account not just study hours and previous grades but also many specific details like the type of study snacks or notebook color. The model ends up fitting the data very closely.
3. **Outcome:** The model predicts test scores with high accuracy for the students in the training data. For instance, it might learn that students who ate a particular snack scored better on the test. This is due to random variations in the training data rather than a real pattern.
4. **New Data:** When the model is used to predict scores for new students, it doesn't perform well. For example, if a new student didn't eat the same snack or use the same color notebook, the predictions might be off. The model fails to generalize because it learned specific quirks from the training data that aren't applicable to all students.

UNIT-2

Regression Analysis in Machine learning

Regression analysis is a statistical method to model the relationship between a dependent (target) and independent (predictor) variables with one or more independent variables.

More specifically, Regression analysis helps us to understand how the value of the dependent variable is changing corresponding to an independent variable when other independent variables are held fixed. It predicts continuous/real values such as temperature, age, salary, price, etc.

Regression:

Regression is a supervised learning technique which helps in finding the correlation between variables and enables us to predict the continuous output variable based on the one or more predictor variables. It is mainly used for prediction, forecasting, time series modeling, and determining the causal-effect relationship between variables.

Some examples of regression can be as:

- Prediction of rain using temperature and other factors
- Determining Market trends
- Prediction of road accidents due to rash driving.

Terminologies Related to the Regression Analysis:

Dependent Variable: The main factor in Regression analysis which we want to predict or understand is called the dependent variable. It is also called **target variable**.

Independent Variable: The factors which affect the dependent variables or which are used to predict the values of the dependent variables are called independent variable, also called as a **predictor**.

Outliers: These are unusual data points that are much higher or lower than most of the data. Outliers can mess up predictions, so they need special attention.

Multicollinearity: This happens when two or more independent variables are very similar. It makes it harder to figure out which factor is more important. It's best to avoid this.

Underfitting and Overfitting:

- **Overfitting:** When the model is too good at remembering the training data but doesn't perform well on new, unseen data.
- **Underfitting:** When the model is too simple and doesn't perform well even on the training data.

Why do we use Regression Analysis?

Below are some other reasons for using Regression analysis:

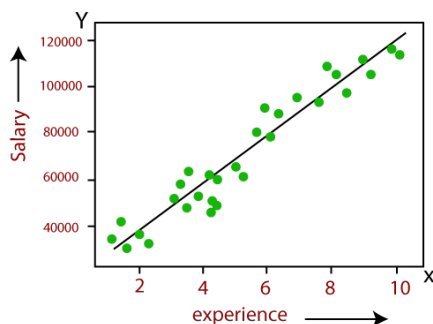
- Regression estimates the relationship between the target and the independent variable.
- It is used to find the trends in data.
- It helps to predict real/continuous values.
- By performing the regression, we can confidently determine the most important factor, the least important factor, and how each factor is affecting the other factors.

Types of Regression

- **Linear Regression**
- **Logistic Regression**
- **Polynomial Regression**
- **Support Vector Regression**
- **Decision Tree Regression**
- **Random Forest Regression**
- **Ridge Regression**
- **Lasso Regression**

Linear Regression:

- Linear regression is a statistical regression method which is used for predictive analysis.
- It is one of the very simple and easy algorithms which works on regression and shows the relationship between the continuous variables.
- It is used for solving the regression problem in machine learning.
- Linear regression shows the linear relationship between the independent variable (X-axis) and the dependent variable (Y-axis), hence called linear regression.
- The relationship between variables in the linear regression model can be explained using the below image. Here we are predicting the salary of an employee on the basis of **the year of experience**.



- Below is the mathematical equation for Linear regression:

1. $Y = aX + b$

Here, Y = dependent variables (target variables),
X= Independent variables (predictor variables),
a and b are the linear coefficients

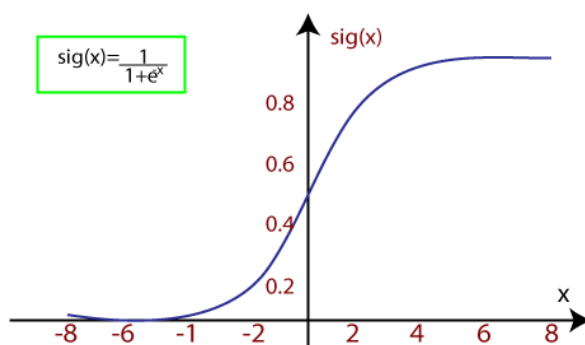
Logistic Regression:

- Logistic regression is another supervised learning algorithm which is used to solve the classification problems. In **classification problems**, we have dependent variables in a binary or discrete format such as 0 or 1.
- Logistic regression algorithm works with the categorical variable such as 0 or 1, Yes or No, True or False, Spam or not spam, etc.
- It is a predictive analysis algorithm which works on the concept of probability.
- Logistic regression uses **sigmoid function** or logistic function which is a complex cost function. This sigmoid function is used to model the data in logistic regression. The function can be represented as:

$$f(x) = \frac{1}{1 + e^{-x}}$$

- $f(x)$ = Output between the 0 and 1 value.
- x = input to the function
- e = base of natural logarithm.

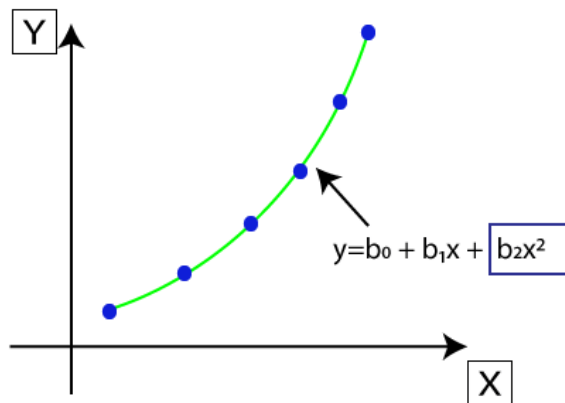
When we provide the input values (data) to the function, it gives the S-curve as follows:



Polynomial Regression:

- Polynomial Regression is a type of regression which models the **non-linear dataset** using a linear model.

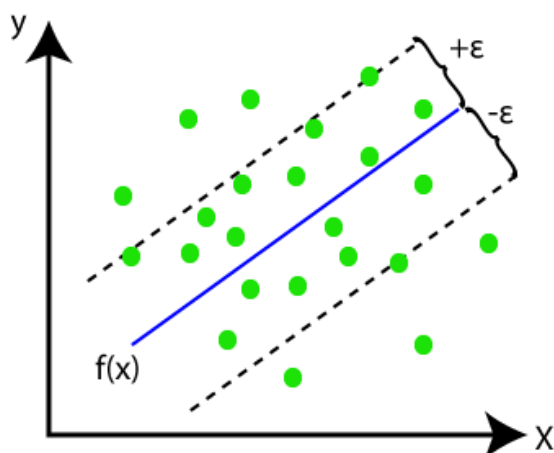
- It is similar to multiple linear regression, but it fits a non-linear curve between the value of x and corresponding conditional values of y .
- **In Polynomial regression, the original features are transformed into polynomial features of given degree and then modeled using a linear model.** Which means the datapoints are best fitted using a polynomial line.



Support Vector Regression:

Support Vector Machine is a supervised learning algorithm which can be used for regression as well as classification problems. So if we use it for regression problems, then it is termed as Support Vector Regression.

The main goal of SVR is to consider the maximum datapoints within the boundary lines and the hyperplane (best-fit line) must contain a maximum number of datapoints.



Here, the blue line is called hyperplane, and the other two lines are known as boundary lines.

Simple Linear Regression in Machine Learning

Simple Linear Regression is a type of Regression algorithms that models the relationship between a dependent variable and a single independent variable. The relationship shown by a Simple Linear Regression model is linear or a sloped straight line, hence it is called Simple Linear Regression.

Simple Linear regression algorithm has mainly two objectives:

- **Model the relationship between the two variables.** Such as the relationship between Income and expenditure, experience and Salary, etc.
- **Forecasting new observations.** Such as Weather forecasting according to temperature, Revenue of a company according to the investments in a year, etc.

Simple Linear Regression Model:

The Simple Linear Regression model can be represented using the below equation

$$Y = a_0 + a_1x + \epsilon$$

a_0 =intercept of regression line.

a_1 =slope of regression line

ϵ = error term

Simple Linear Regression Assumptions:

1.Linearity: The relationship between the independent and dependent variable should be a straight line.

2.Normality of Residuals: The errors (residuals) should follow a normal distribution.

3.No Autocorrelation: This means the residuals should not be correlated with each other. This is especially important in time series data.

Simple linear regression model building

1.Import Libraries: Use libraries like pandas (for data handling), numpy (for mathematical operations), and sklearn (for regression).

```
import pandas as pd
```

```
import numpy as np
```

```
from sklearn.linear_model import LinearRegression
```

2.Load Dataset: Load your data (e.g., CSV file) into a DataFrame.

```
data = pd.read_csv('data.csv')
```

3.Define Variables: Choose your independent (X) and dependent (Y) variables.

```
X = data[['independent_variable']] # Independent variable
```

```
Y = data['dependent_variable'] # Dependent variable
```

4.Split Data: Split the data into training and testing sets (usually 70-80% for training).

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2)
```

5.Create and Train Model: Create a linear regression model and train it on the training data.

```
model = LinearRegression()
```

```
model.fit(X_train, Y_train)
```

6.Make Predictions: Use the trained model to predict on the test data.

```
Y_pred = model.predict(X_test)
```

7.Evaluate the Model: Check how well the model performs using metrics like R-squared or Mean Squared Error (MSE).

```
from sklearn.metrics import mean_squared_error
```

```
mse = mean_squared_error(Y_test, Y_pred)
```

```
print(f'Mean Squared Error: {mse}')
```

8.Visualize Results: Plot the regression line and data points for better understanding.

```
import matplotlib.pyplot as plt
```

```
plt.scatter(X_test, Y_test, color='blue')
```

```
plt.plot(X_test, Y_pred, color='red')
```

```
plt.show()
```

Multiple Linear Regression

Multiple Linear Regression is an extension of simple linear regression that models the relationship between one **dependent variable** and two or more **independent variables**. The goal is to understand how the independent variables impact the dependent variable.

Equation: The general form of a multiple linear regression model is:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n + \epsilon$$

Where:

- y is the **dependent variable** (the variable we want to predict).
- $x_1, x_2, \dots, x_n$ are the **independent variables** (predictors).
- b_0 is the **intercept** (value of y when all x 's are 0).
- $b_1, b_2, \dots, b_n$ are the **coefficients** (showing the impact of each independent variable on the dependent variable).
- ϵ is the **error term**, accounting for variability not explained by the independent variables.

Assumptions for Multiple Linear Regression:

- A **linear relationship** should exist between the Target and predictor variables.
- The regression residuals must be **normally distributed**.
- MLR assumes little or **no multicollinearity** (correlation between the independent variable) in data.

Feature Selection

Feature Selection in machine learning refers to the process of selecting the most relevant and important features (variables) from your dataset to improve the model's performance, reduce overfitting, and make the model easier to interpret. It helps to focus on the most informative parts of the data while discarding irrelevant or redundant features.

Why Feature Selection is Important:

1. **Improves Model Accuracy:** By removing irrelevant or redundant features, feature selection can improve the accuracy and performance of your model.
2. **Reduces Overfitting:** Less irrelevant data reduces the noise and the chances of overfitting, where the model learns patterns specific to the training data.
3. **Reduces Training Time:** Fewer features lead to simpler models, which require less computational power and train faster.
4. **Simplifies the Model:** Reduces model complexity, making it easier to interpret and deploy.

Types of Feature Selection Methods:

Feature selection methods can be divided into three categories:

1. Filter Methods:

- **How it Works:** These methods apply statistical techniques to select features independently of the model. They rank features based on statistical metrics and select the highest-ranked features.
- **Techniques:**
 - **Correlation Coefficient:** Measures how features are correlated with the target variable. Features with high correlation are selected.
 - **Chi-Square Test:** Used for categorical features, measures the association between the feature and the target variable.
 - **ANOVA (Analysis of Variance):** Measures the difference between the means of different groups. Useful for continuous target variables.

2. Wrapper Methods:

- **How it Works:** These methods use the machine learning model itself to evaluate the importance of features. They involve training a model on different subsets of features and selecting the subset that yields the best performance.
- **Techniques:**
 - **Forward Selection:** Start with no features and add them one by one, evaluating the model at each step, and keeping the ones that improve performance.
 - **Backward Elimination:** Start with all features, remove them one by one, and check model performance. Remove features that don't improve or harm performance.
 - **Recursive Feature Elimination (RFE):** Starts with all features and recursively removes the least important ones based on model performance.

3. Embedded Methods:

- **How it Works:** These methods perform feature selection during the model training process. Some models have built-in feature selection capabilities where important features are identified as part of the learning algorithm.
- **Techniques:**
 - **Lasso Regression (L1 Regularization):** Adds a penalty term to the model for having too many features, forcing less important feature coefficients to zero.
 - **Ridge Regression (L2 Regularization):** Reduces the magnitude of less important feature coefficients.
 - **Decision Trees and Random Forest:** These models automatically rank features by importance based on how they reduce impurity at each split.

How to Choose a Feature Selection Method:

- **Filter Methods:** Good when you have a large dataset and need a quick and simple method to reduce dimensionality.
- **Wrapper Methods:** Best when accuracy is important and computational resources are not a limitation.
- **Embedded Methods:** Suitable for when you're using models like decision trees, Lasso, or Ridge, as they perform feature selection internally.

Dimensionality Reduction

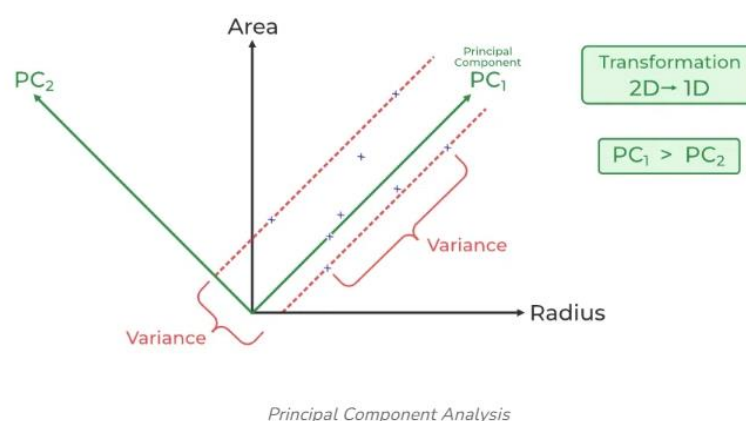
Dimensionality Reduction in machine learning is the process of reducing the number of features (variables) in a dataset while preserving its essential information. It is particularly useful when dealing with high-dimensional data, as it can help improve model performance, reduce overfitting, and decrease computational costs.

Why Dimensionality Reduction is Important:

1. **Improves Model Performance:** Reducing the number of features can lead to better model performance by eliminating noise and irrelevant information.
2. **Reduces Overfitting:** Fewer dimensions can help the model generalize better to unseen data.
3. **Decreases Computation Time:** Fewer features mean faster training and testing times.
4. **Enhances Visualization:** It allows for easier visualization of data in lower dimensions, aiding in understanding and interpreting the data.

Principal Component Analysis (PCA)

- **Principal Component Analysis (PCA)** is a statistical procedure that uses an orthogonal transformation that converts a set of correlated variables to a set of uncorrelated variables. PCA is the most widely used tool in exploratory data analysis and in machine learning for predictive models. Moreover,
- Principal Component Analysis (PCA) is an unsupervised learning algorithm technique used to examine the interrelations among a set of variables. It is also known as a general factor analysis where regression determines a line of best fit.
- The main goal of Principal Component Analysis (PCA) is to reduce the dimensionality of a dataset while preserving the most important patterns or relationships between the variables without any prior knowledge of the target variables.



Steps in PCA (Principal Component Analysis)

Step 1: Standardization

First, we need to standardize our dataset to ensure that each variable has a mean of 0 and a standard deviation of 1.

$$Z = \frac{X - \mu}{\sigma}$$

Here,

- μ is the mean of independent features $\mu = \{\mu_1, \mu_2, \dots, \mu_m\}$
- σ is the [standard deviation](#) of independent features $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$

Step2: Covariance Matrix Computation

Covariance measures the strength of joint variability between two or more variables, indicating how much they change in relation to each other. To find the covariance we can use the formula:

$$\text{cov}(x1, x2) = \frac{\sum_{i=1}^n (x1_i - \bar{x1})(x2_i - \bar{x2})}{n-1}$$

The value of covariance can be positive, negative, or zeros.

- Positive: As the x1 increases x2 also increases.
- Negative: As the x1 increases x2 also decreases.
- Zeros: No direct relation

Step 3: Compute Eigenvalues and Eigenvectors of Covariance Matrix to Identify Principal Components

Let A be a square nXn matrix and X be a non-zero vector for which $AX = \lambda X$

for some scalar values λ . then λ is known as the eigenvalue of matrix A and X is known as the eigenvector of matrix A for the corresponding eigenvalue.

It can also be written as :

$$AX - \lambda X = 0$$

$$(A - \lambda I)X = 0$$

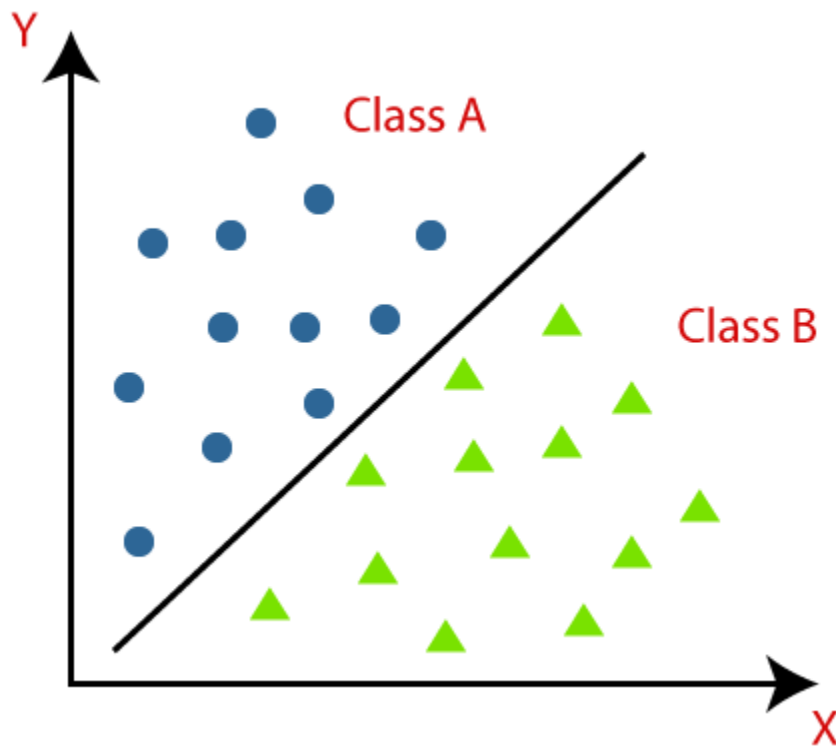
where I is the identity matrix of the same shape as matrix A.

UNIT-3

What is the Classification Algorithm?

The Classification algorithm is a Supervised Learning technique that is used to identify the category of new observations on the basis of training data. In Classification, a program learns from the given dataset or observations and then classifies new observation into a number of classes or groups. Such as, **Yes or No, 0 or 1, Spam or Not Spam, cat or dog**, etc. Classes can be called as targets/labels or categories.

Classification algorithms can be better understood using the below diagram. In the below diagram, there are two classes, class A and Class B. These classes have features that are similar to each other and dissimilar to other classes.



The algorithm which implements the classification on a dataset is known as a classifier. There are two types of Classifications:

- **Binary Classifier:** If the classification problem has only two possible outcomes, then it is called as Binary Classifier.
Examples: YES or NO, MALE or FEMALE, SPAM or NOT SPAM, CAT or DOG, etc.
- **Multi-class Classifier:** If a classification problem has more than two outcomes, then it is called as Multi-class Classifier.
Example: Classifications of types of crops, Classification of types of music.

Classification Algorithms

There are various types of **classifiers algorithms**. Some of them are :

Linear Classifiers

Linear models create a linear decision boundary between classes. They are simple and computationally efficient. Some of the linear **classification** models are as follows:

- Logistic Regression
- Support Vector Machines having kernel = 'linear'
- Single-layer Perceptron
- Stochastic Gradient Descent (SGD) Classifier

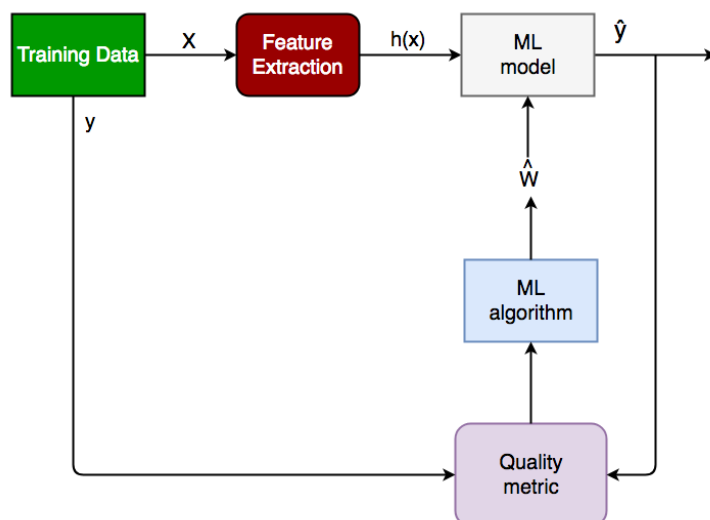
Non-linear Classifiers

Non-linear models create a non-linear decision boundary between classes. They can capture more complex relationships between the input features and the target variable. Some of the non-linear **classification** models are as follows:

- K-Nearest Neighbours
- Kernel SVM
- Naive Bayes
- Decision Tree Classification
- Ensemble learning classifiers:
 - Random Forests,
 - AdaBoost,
 - Bagging Classifier,
 - Voting Classifier,
 - ExtraTrees Classifier
- Multi-layer Artificial Neural Networks

How does Classification Machine Learning Work?

The basic idea behind classification is to train a model on a labeled dataset, where the input data is associated with their corresponding output labels, to learn the patterns and relationships between the input data and output labels. Once the model is trained, it can be used to predict the output labels for new unseen data.



The classification process typically involves the following steps:

Understanding the problem

Before getting started with classification, it is important to understand the problem you are trying to solve. What are the class labels you are trying to predict? What is the relationship between the input data and the class labels?

Suppose we have to predict whether a patient has a certain disease or not, on the basis of 7 independent variables, called features. This means, there can be only two possible outcomes:

- The patient has the disease, which means “**True**”.
- The patient has no disease. which means “**False**”.

This is a binary classification problem.

Data preparation

Once you have a good understanding of the problem, the next step is to prepare your data. This includes collecting and preprocessing the data and splitting it into training, validation, and test sets. In this step, the data is cleaned, preprocessed, and transformed into a format that can be used by the classification algorithm.

- **X**: It is the independent feature, in the form of an $N \times M$ matrix. N is the no. of observations and M is the number of features.
- **y**: An N vector corresponding to predicted classes for each of the N observations.

Feature Extraction

The relevant features or attributes are extracted from the data that can be used to differentiate between the different classes.

Suppose our input X has 7 independent features, having only 5 features influencing the label or target values remaining 2 are negligibly or not correlated, then we will use only these 5 features only for the model training.

Model Selection

There are many different models that can be used for classification, including **logistic regression, decision trees, support vector machines (SVM), or neural networks**. It is important to select a model that is appropriate for your problem, taking into account the size and complexity of your data, and the computational resources you have available.

Model Training

Once you have selected a model, the next step is to train it on your training data. This involves adjusting the parameters of the model to minimize the error between the predicted class labels and the actual class labels for the training data.

Model Evaluation

Evaluating the model: After training the model, it is important to evaluate its performance on a validation set. This will give you a good idea of how well the model is likely to perform on new, unseen data.

Fine-tuning the model

If the model's performance is not satisfactory, you can fine-tune it by adjusting the parameters, or trying a different model.

Deploying the model

Finally, once we are satisfied with the performance of the model, we can deploy it to make predictions on new data. It can be used for real world problem.

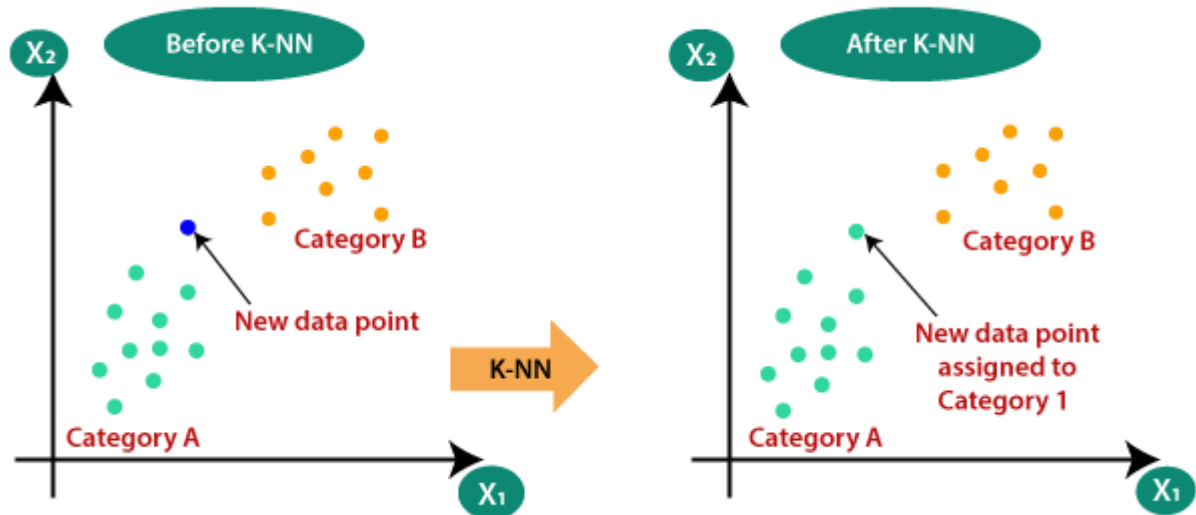
K-Nearest Neighbour (KNN) Algorithm for Machine Learning

- K-Nearest Neighbour is one of the simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.
- K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.
- **Example:** Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



Why do we need a K-NN Algorithm?

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x_1 , so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



How does K-NN work?

The K-NN working can be explained on the basis of the below algorithm:

- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.

Suppose we have a new data point and we need to put it in the required category. Consider the below image:

How to select the value of K in the K-NN Algorithm?

Below are some points to remember while selecting the value of K in the K-NN algorithm:

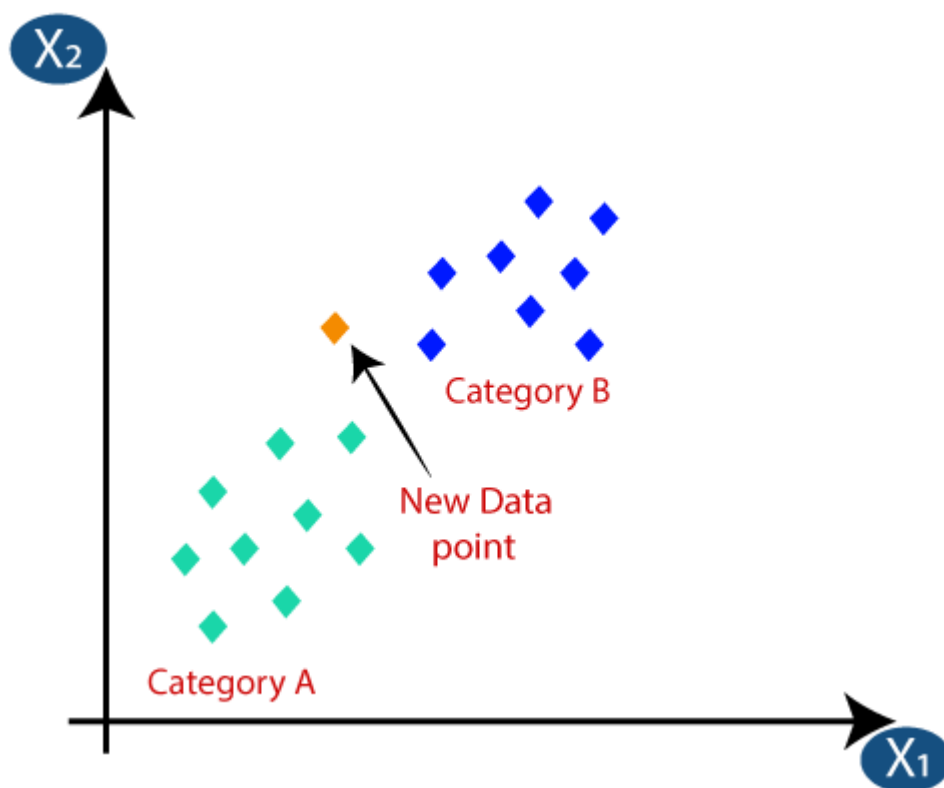
- There is no particular way to determine the best value for " K ", so we need to try some values to find the best out of them. The most preferred value for K is 5.
- A very low value for K such as $K=1$ or $K=2$, can be noisy and lead to the effects of outliers in the model.
- Large values for K are good, but it may find some difficulties.

Advantages of KNN Algorithm:

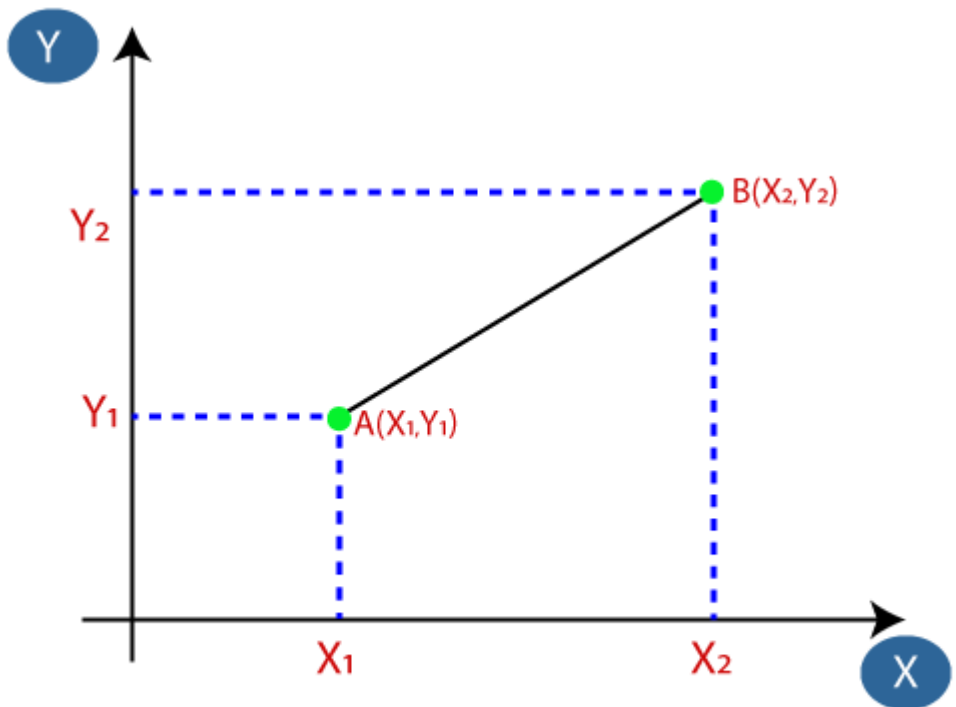
- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

Disadvantages of KNN Algorithm:

- Always needs to determine the value of K which may be complex some time.
- The computation cost is high because of calculating the distance between the data points for all the training samples.

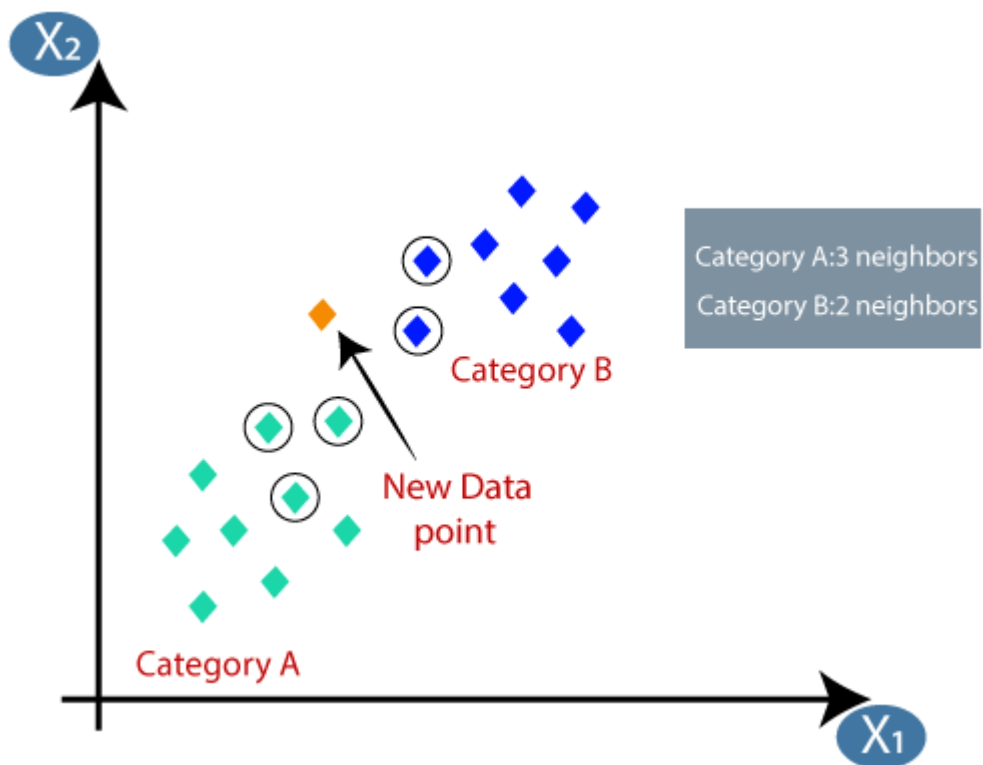


- Firstly, we will choose the number of neighbors, so we will choose the $k=5$.
- Next, we will calculate the **Euclidean distance** between the data points. The Euclidean distance is the distance between two points, which we have already studied in geometry. It can be calculated as:



Euclidean Distance between A_1 and $B_2 = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$

- By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:



- As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

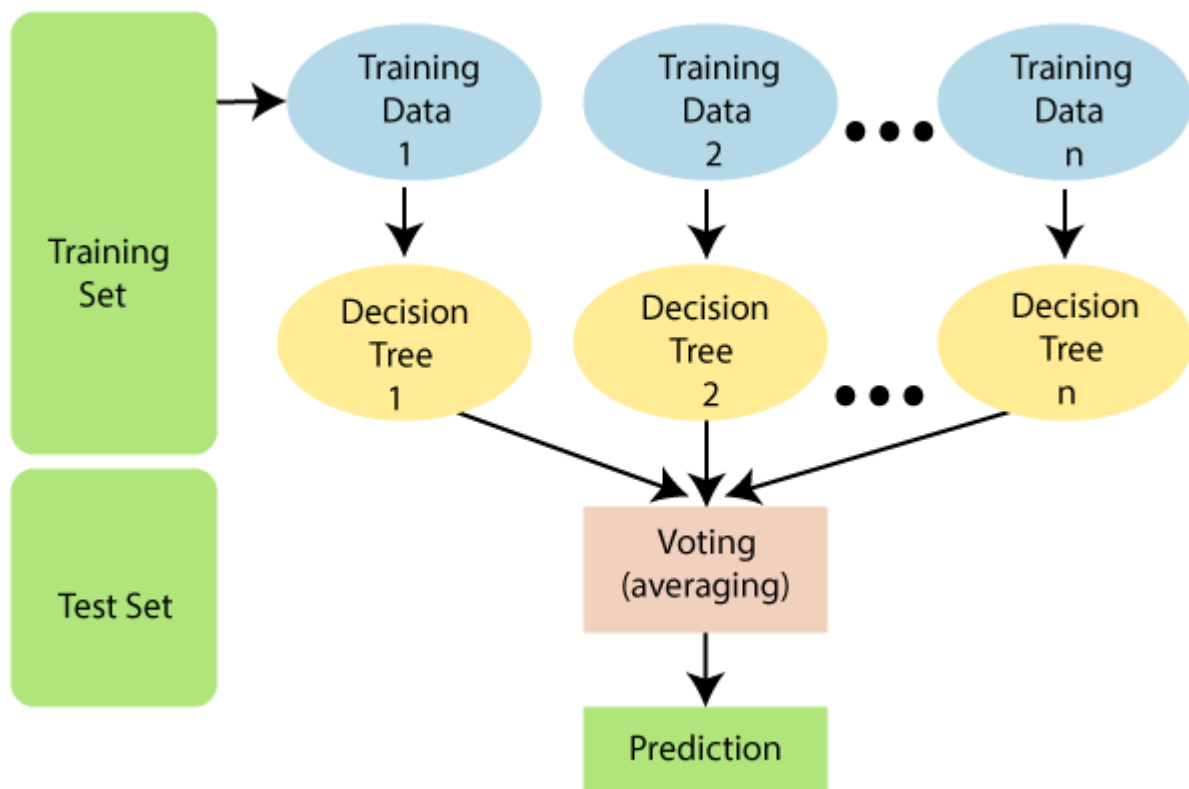
Random Forest Algorithm

Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique. It can be used for both Classification and Regression problems in ML. It is based on the concept of **ensemble learning**, which is a process of *combining multiple classifiers to solve a complex problem and to improve the performance of the model*.

As the name suggests, **"Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset."** Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.

The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

The below diagram explains the working of the Random Forest algorithm:



Assumptions for Random Forest

Since the random forest combines multiple trees to predict the class of the dataset, it is possible that some decision trees may predict the correct output, while others may not. But together, all the trees predict the correct output. Therefore, below are two assumptions for a better Random forest classifier:

- There should be some actual values in the feature variable of the dataset so that the classifier can predict accurate results rather than a guessed result.
- The predictions from each tree must have very low correlations.

Why use Random Forest?

Below are some points that explain why we should use the Random Forest algorithm:

- It takes less training time as compared to other algorithms.
- It predicts output with high accuracy, even for the large dataset it runs efficiently.
- It can also maintain accuracy when a large proportion of data is missing.

How does Random Forest algorithm work?

Random Forest works in two-phase first is to create the random forest by combining N decision tree, and second is to make predictions for each tree created in the first phase.

The Working process can be explained in the below steps and diagram:

Step-1: Select random K data points from the training set.

Step-2: Build the decision trees associated with the selected data points (Subsets).

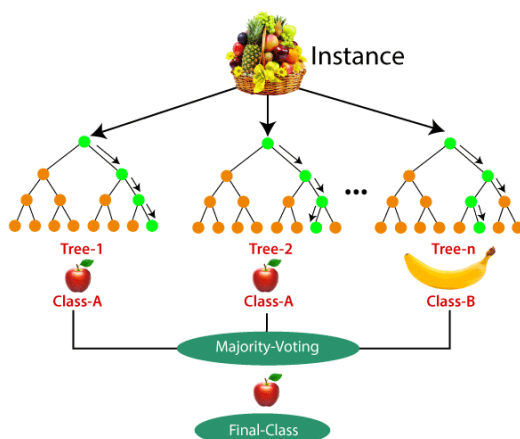
Step-3: Choose the number N for decision trees that you want to build.

Step-4: Repeat Step 1 & 2.

Step-5: For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

The working of the algorithm can be better understood by the below example:

Example: Suppose there is a dataset that contains multiple fruit images. So, this dataset is given to the Random forest classifier. The dataset is divided into subsets and given to each decision tree. During the training phase, each decision tree produces a prediction result, and when a new data point occurs, then based on the majority of results, the Random Forest classifier predicts the final decision. Consider the below image:



Applications of Random Forest

There are mainly four sectors where Random forest mostly used:

1. **Banking:** Banking sector mostly uses this algorithm for the identification of loan risk.
2. **Medicine:** With the help of this algorithm, disease trends and risks of the disease can be identified.
3. **Land Use:** We can identify the areas of similar land use by this algorithm.
4. **Marketing:** Marketing trends can be identified using this algorithm.

Advantages of Random Forest

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of Random Forest

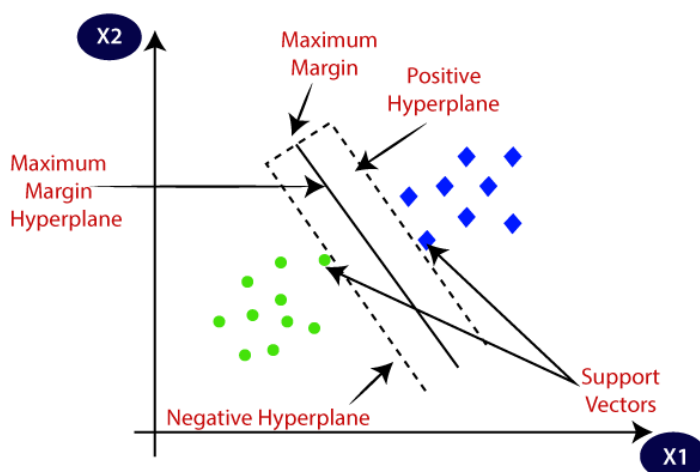
- Although random forest can be used for both classification and regression tasks, it is not more suitable for Regression tasks.

Support Vector Machine Algorithm

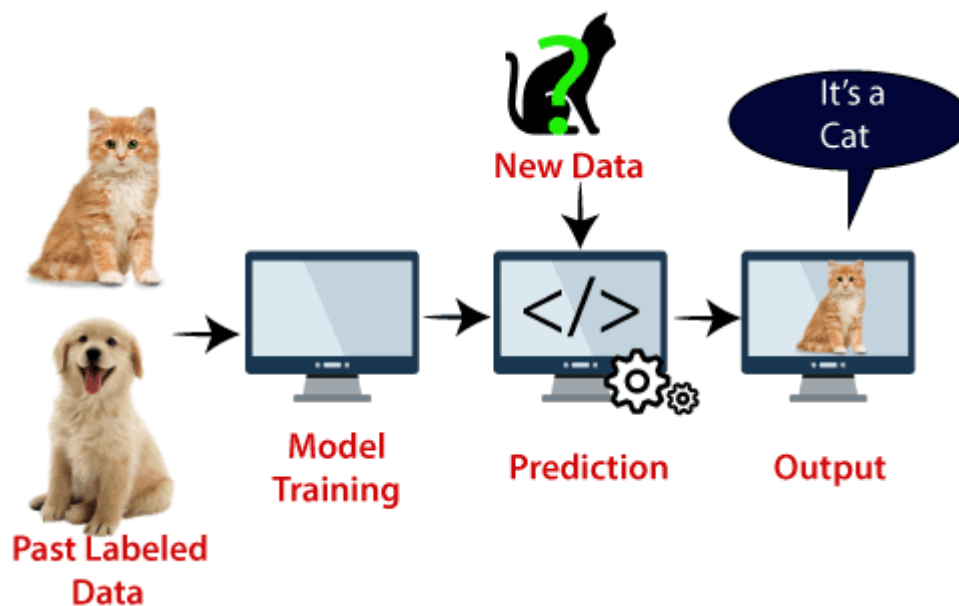
Support Vector Machine or SVM is one of the most popular Supervised Learning algorithms, which is used for Classification as well as Regression problems. However, primarily, it is used for Classification problems in Machine Learning.

The goal of the SVM algorithm is to create the best line or decision boundary that can segregate n-dimensional space into classes so that we can easily put the new data point in the correct category in the future. This best decision boundary is called a hyperplane.

SVM chooses the extreme points/vectors that help in creating the hyperplane. These extreme cases are called as support vectors, and hence algorithm is termed as Support Vector Machine. Consider the below diagram in which there are two different categories that are classified using a decision boundary or hyperplane:



Example: SVM can be understood with the example that we have used in the KNN classifier. Suppose we see a strange cat that also has some features of dogs, so if we want a model that can accurately identify whether it is a cat or dog, so such a model can be created by using the SVM algorithm. We will first train our model with lots of images of cats and dogs so that it can learn about different features of cats and dogs, and then we test it with this strange creature. So as support vector creates a decision boundary between these two data (cat and dog) and choose extreme cases (support vectors), it will see the extreme case of cat and dog. On the basis of the support vectors, it will classify it as a cat. Consider the below diagram:



SVM algorithm can be used for **Face detection, image classification, text categorization**, etc.

Types of SVM

SVM can be of two types:

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a single straight line, then such data is termed as linearly separable data, and classifier is used called as Linear SVM classifier.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a straight line, then such data is termed as non-linear data and classifier used is called as Non-linear SVM classifier.

Hyperplane and Support Vectors in the SVM algorithm:

Hyperplane: There can be multiple lines/decision boundaries to segregate the classes in n-dimensional space, but we need to find out the best decision boundary that helps to classify the data points. This best boundary is known as the hyperplane of SVM.

The dimensions of the hyperplane depend on the features present in the dataset, which means if there are 2 features (as shown in image), then hyperplane will be a straight line. And if there are 3 features, then hyperplane will be a 2-dimension plane.

We always create a hyperplane that has a maximum margin, which means the maximum distance between the data points.

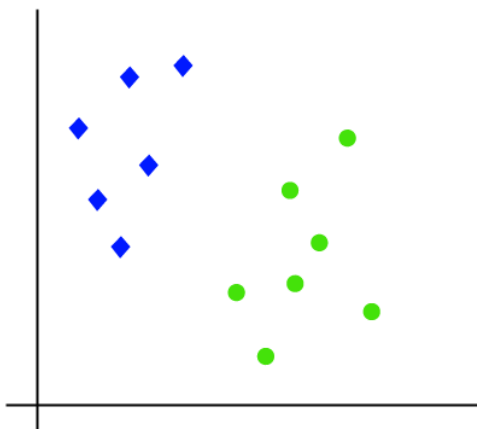
Support Vectors:

The data points or vectors that are the closest to the hyperplane and which affect the position of the hyperplane are termed as Support Vector. Since these vectors support the hyperplane, hence called a Support vector.

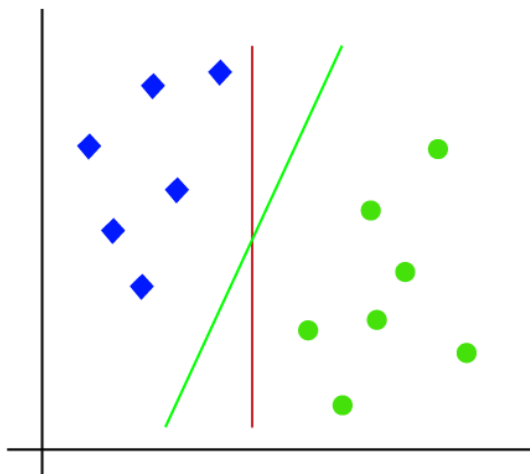
How does SVM works?

Linear SVM:

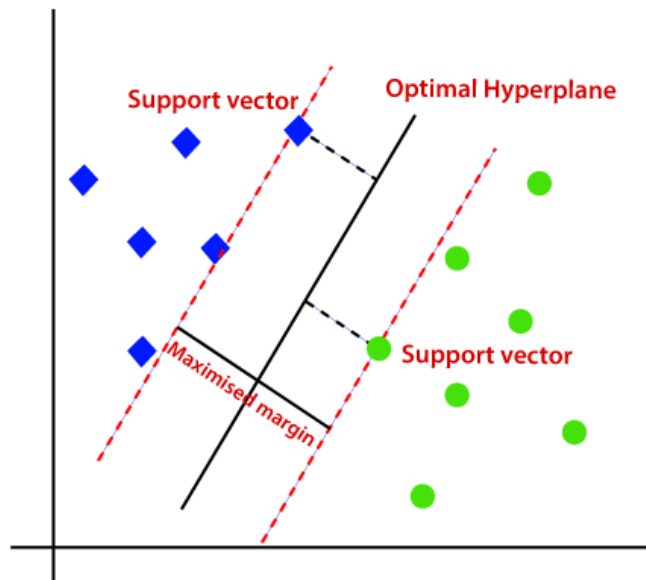
The working of the SVM algorithm can be understood by using an example. Suppose we have a dataset that has two tags (green and blue), and the dataset has two features x_1 and x_2 . We want a classifier that can classify the pair(x_1 , x_2) of coordinates in either green or blue. Consider the below image:



So as it is 2-d space so by just using a straight line, we can easily separate these two classes. But there can be multiple lines that can separate these classes. Consider the below image:

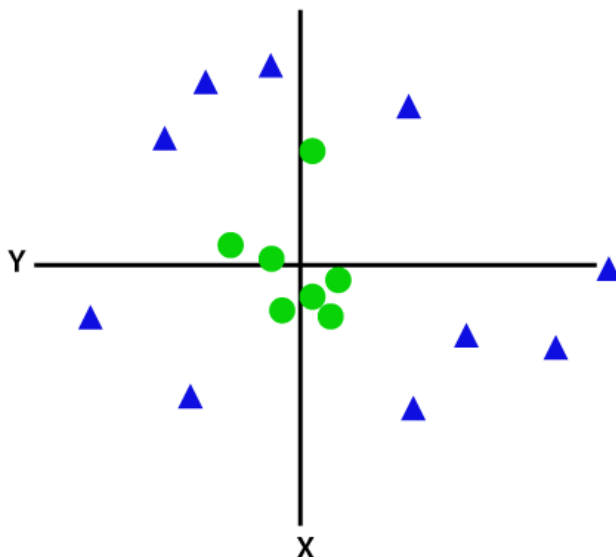


Hence, the SVM algorithm helps to find the best line or decision boundary; this best boundary or region is called as a **hyperplane**. SVM algorithm finds the closest point of the lines from both the classes. These points are called support vectors. The distance between the vectors and the hyperplane is called as **margin**. And the goal of SVM is to maximize this margin. The **hyperplane** with maximum margin is called the **optimal hyperplane**.



Non-Linear SVM:

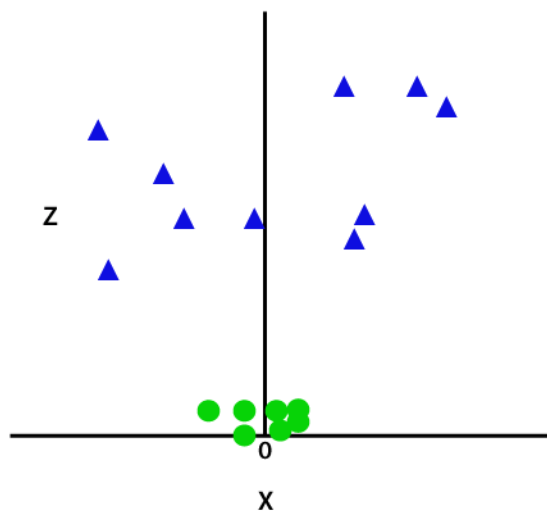
If data is linearly arranged, then we can separate it by using a straight line, but for non-linear data, we cannot draw a single straight line. Consider the below image:



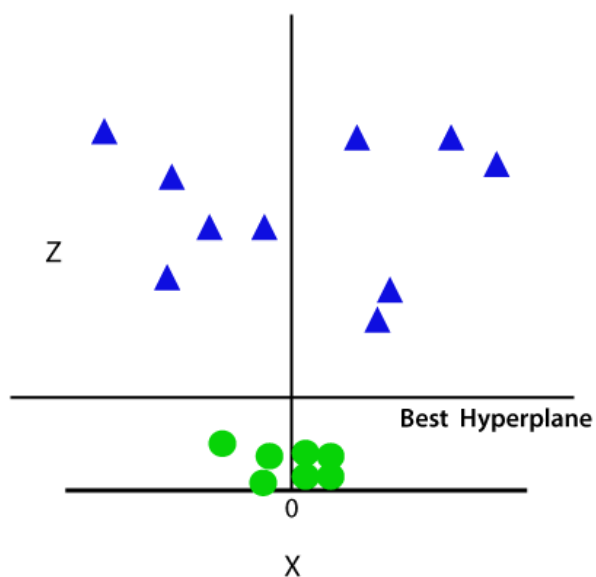
So to separate these data points, we need to add one more dimension. For linear data, we have used two dimensions x and y , so for non-linear data, we will add a third dimension z . It can be calculated as:

$$z = x^2 + y^2$$

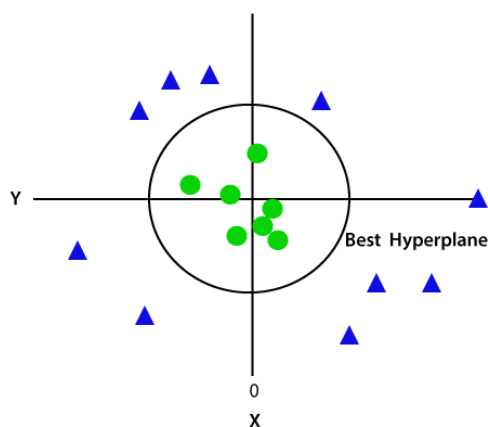
By adding the third dimension, the sample space will become as below image:



So now, SVM will divide the datasets into classes in the following way. Consider the below image:



Since we are in 3-d Space, hence it is looking like a plane parallel to the x-axis. If we convert it in 2d space with $z=1$, then it will become as:



Hence we get a circumference of radius 1 in case of non-linear data.

Kernel Method in SVMs

Support Vector Machines (SVMs) use kernel methods to transform the input data into a higher-dimensional feature space, which makes it simpler to distinguish between classes or generate predictions. Kernel approaches in SVMs work on the fundamental principle of implicitly mapping input data into a higher-dimensional feature space without directly computing the coordinates of the data points in that space.

The kernel function in SVMs is essential in determining the decision boundary that divides the various classes. In order to calculate the degree of similarity between any two points in the feature space, the kernel function computes their dot product.

Characteristics of Kernel Function

Kernel functions used in machine learning, including in SVMs (Support Vector Machines), have several important characteristics, including:

- **Mercer's condition:** A kernel function must satisfy Mercer's condition to be valid. This condition ensures that the kernel function is positive semi definite, which means that it is always greater than or equal to zero.
- **Positive definiteness:** A kernel function is positive definite if it is always greater than zero except for when the inputs are equal to each other.
- **Non-negativity:** A kernel function is non-negative, meaning that it produces non-negative values for all inputs.
- **Symmetry:** A kernel function is symmetric, meaning that it produces the same value regardless of the order in which the inputs are given.
- **Reproducing property:** A kernel function satisfies the reproducing property if it can be used to reconstruct the input data in the feature space.
- **Smoothness:** A kernel function is said to be smooth if it produces a smooth transformation of the input data into the feature space.
- **Complexity:** The complexity of a kernel function is an important consideration, as more complex kernel functions may lead to over fitting and reduced generalization performance.

Major Kernel Function in Support Vector Machine

In Support Vector Machines (SVMs), there are several types of kernel functions that can be used to map the input data into a higher-dimensional feature space. The choice of kernel function depends on the specific problem and the characteristics of the data.

Here are some most commonly used kernel functions in SVMs:

Linear Kernel

A linear kernel is a type of kernel function used in machine learning, including in SVMs (Support Vector Machines). It is the simplest and most commonly used kernel function, and it defines the dot product between the input vectors in the original feature space.

The linear kernel can be defined as:

1. $K(x, y) = x \cdot y$

Where x and y are the input feature vectors. The dot product of the input vectors is a measure of their similarity or distance in the original feature space.

Polynomial Kernel

A particular kind of kernel function utilised in machine learning, such as in SVMs, is a polynomial kernel (Support Vector Machines). It is a nonlinear kernel function that employs polynomial functions to transfer the input data into a higher-dimensional feature space.

One definition of the polynomial kernel is:

Where x and y are the input feature vectors, c is a constant term, and d is the degree of the polynomial, $K(x, y) = (x \cdot y + c)^d$. The constant term is added to, and the dot product of the input vectors elevated to the degree of the polynomial.

Gaussian (RBF) Kernel

The Gaussian kernel, also known as the radial basis function (RBF) kernel, is a popular kernel function used in machine learning, particularly in SVMs (Support Vector Machines). It is a nonlinear kernel function that maps the input data into a higher-dimensional feature space using a Gaussian function.

The Gaussian kernel can be defined as:

1. $K(x, y) = \exp(-\gamma \cdot \|x - y\|^2)$

Where x and y are the input feature vectors, γ is a parameter that controls the width of the Gaussian function, and $\|x - y\|^2$ is the squared Euclidean distance between the input vectors.

Laplace Kernel

The Laplacian kernel, also known as the Laplace kernel or the exponential kernel, is a type of kernel function used in machine learning, including in SVMs (Support Vector Machines). It is a non-parametric kernel that can be used to measure the similarity or distance between two input feature vectors.

The Laplacian kernel can be defined as:

1. $K(x, y) = \exp(-\gamma \cdot \|x - y\|)$

Where x and y are the input feature vectors, γ is a parameter that controls the width of the Laplacian function, and $\|x - y\|$ is the L1 norm or Manhattan distance between the input vectors.

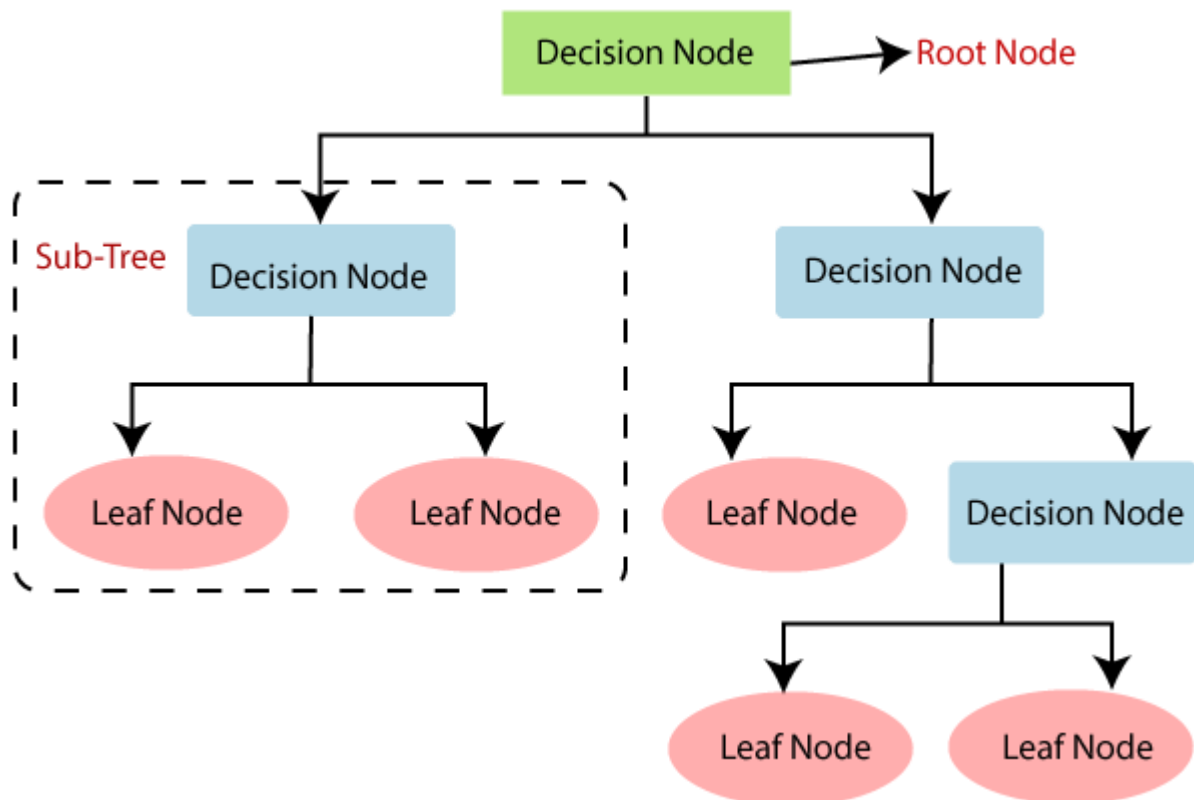
Properties of SVM

- Flexibility in choosing a similarity function
- Sparseness of solution when dealing with large data sets
- only support vectors are used to specify the separating hyperplane
- Ability to handle large feature spaces
- complexity does not depend on the dimensionality of the feature space
- Overfitting can be controlled by soft margin approach
- **Nice math property:** a simple convex optimization problem which is guaranteed to converge to a single global solution
- Feature Selection

Decision Tree Classification Algorithm

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome**.
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node**. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the **CART algorithm**, which stands for **Classification and Regression Tree algorithm**.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- Below diagram explains the general structure of a decision tree:

Note: A decision tree can contain categorical data (YES/NO) as well as numeric data.



Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

❓ **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

❓ **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

❓ **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

❓ **Branch/Sub Tree:** A tree formed by splitting the tree.

❓ **Pruning:** Pruning is the process of removing the unwanted branches from the tree.

❓ **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

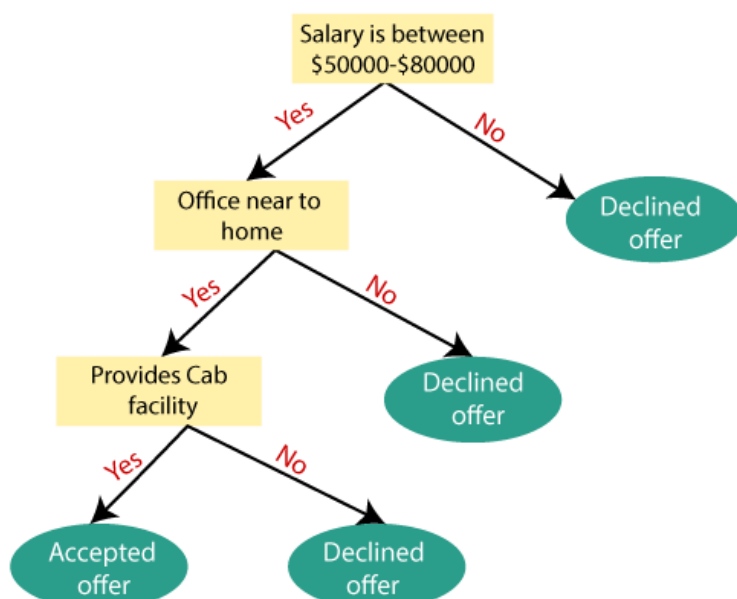
How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM)**.
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:



The ID3 (Iterative Dichotomiser 3) Algorithm in Machine Learning is a popular decision tree algorithm used to classify data. It works by selecting the attribute that provides the maximum information gain for splitting the data.

ID3 Algorithm Steps:

The ID3 (Iterative Dichotomiser 3) algorithm is pretty easy and a powerful algorithm used to construct decision trees. The algorithm involves several key steps:

1. **Selecting the Best Attribute:** Begin by selecting the best attribute that splits the data into subsets. This is done using a metric named as information gain, which measures how well an attribute separates the data into groups based on the target attribute.
2. **Tree Construction:** Use the best attribute as a decision node and branch off from it for each possible value of the attribute. This process is mainly for partitioning the data.
3. **Recursive Splitting:** Repeat the process for each branch using the remaining attributes. Stop if all instances in a branch are the same or no more attributes are available.
4. **Pruning (Optional):** Simplify the tree by removing branches that have little effect on the decision-making process to reduce overfitting and improve the model's generalizability.

How does ID3 Algorithm Works?

The ID3 algorithm builds a decision tree by selecting the attribute that separates the data into different classes in the best way possible. Here's a step-by-step overview of how the algorithm works:

- **Start with the Entire Dataset:** The algorithm begins by considering the entire dataset as a whole.
- **Calculate Entropy:** Entropy measures the level of uncertainty or impurity in a dataset. It helps to determine how well a dataset is mixed or split between different classes. In decision trees, entropy is used to calculate the best feature to split the data on.

The formula for entropy is:

$$E(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

- where p_i is the proportion of examples in class i ,
- n is the total number of classes.
- **Determine Information Gain for Each Attribute:** Information Gain is the reduction in entropy achieved by splitting the data based on an attribute. The attribute with the

highest Information Gain is selected for the split. The formula for Information Gain is

$$IG(S, A) = E(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} E(S_v)$$

Where:

- $E(S)$ is the entropy of the entire dataset S ,
- S_v is the subset of S where attribute A has the value v ,
- $|S_v|$ is the size of the subset S_v ,
- $|S|$ is the size of the dataset S ,
- $E(S_v)$ is the entropy of the subset S_v .
- **Split the Dataset:** The dataset is split based on the chosen attribute, and the process is repeated for each subset until all data points are perfectly classified, or no further splits can be made.
- **Create Leaf Nodes:** Once the data is fully classified, the nodes at the ends of the branches become leaf nodes, representing the final decision or classification.

Mathematical Concepts of ID3 Algorithm

The ID3 algorithm depends mainly on two main mathematical concepts: Entropy and Information Gain.

1. Entropy

Entropy measures the level of uncertainty in a dataset. In decision trees, it quantifies the randomness or impurity present. Low entropy means most data points belong to one class, while high entropy shows a mix of classes. For example, if all data points in a dataset are classified as "Yes," the entropy will be zero due to no uncertainty. On the other hand, a 50/50 split between "Yes" and "No" indicates maximum entropy due to higher uncertainty.

2. Information Gain

Information Gain measures how well an attribute separates the data. It is calculated as the difference between the entropy of the dataset before the split and the weighted average of the entropy after the split.

For example, in a dataset where splitting based on the "Outlook" attribute reduces the entropy the most, the ID3 algorithm will select "Outlook" as the root node of the decision tree.

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm**.
- For more class labels, the computational complexity of the decision tree may increase.

Ensemble Methods

Ensemble methods in machine learning combine predictions from multiple models to improve performance, accuracy, and robustness. Popular ensemble methods include **Bagging**, **Boosting**, **AdaBoost**, and **XGBoost**.

1. Bagging (Bootstrap Aggregating)

Key Idea: Combine predictions from multiple models trained independently on random subsets of the data.

- **Steps:**
 1. Create multiple subsets of the dataset by random sampling **with replacement** (bootstrap sampling).
 2. Train a separate model (e.g., decision trees) on each subset.
 3. Combine predictions (e.g., average for regression or majority vote for classification).
- **Advantages:**
 - Reduces variance (prevents overfitting).
 - Works well with high-variance models like decision trees.
- **Example: Random Forest**
 - Random Forest is an extension of Bagging where each tree also selects a random subset of features for splitting.

2. Boosting

Key Idea: Build models sequentially, where each model tries to correct the errors of the previous one.

- **Steps:**
 1. Start with a weak model (e.g., a shallow decision tree).
 2. Train the next model to focus on the errors (misclassified examples) from the previous model.
 3. Combine all models' predictions, usually with weighted voting.
- **Advantages:**
 - Reduces bias and variance.
 - Works well with weak learners, like shallow decision trees.

- **Drawbacks:**
 - More prone to overfitting compared to Bagging.
 - Can be computationally expensive.

3. AdaBoost (Adaptive Boosting)

Key Idea: A type of Boosting where each model focuses on correcting the mistakes of the previous one by adjusting the weights of misclassified samples.

- **Steps:**
 1. Initialize equal weights for all samples.
 2. Train a weak learner (e.g., a decision stump).
 3. Increase weights for misclassified samples so the next model focuses more on them.
 4. Combine all weak learners' predictions using a weighted sum.
- **Advantages:**
 - Simple and effective for many tasks.
 - Handles outliers better than regular boosting.
- **Drawbacks:**
 - Sensitive to noise and outliers because it gives higher weights to difficult examples.

4. XGBoost (Extreme Gradient Boosting)

Key Idea: An optimized version of Gradient Boosting that uses advanced techniques to improve speed and performance.

- **Steps:**
 1. Build a series of decision trees using Gradient Boosting.
 2. Use regularization techniques (L1 and L2) to prevent overfitting.
 3. Employ techniques like parallel processing, sparse matrix optimization, and early stopping for faster training.
- **Advantages:**
 - Faster and more efficient than traditional Boosting.
 - Regularization helps to avoid overfitting.
 - Highly customizable for complex datasets.
- **Applications:**
 - Often used in machine learning competitions (e.g., Kaggle) because of its performance and versatility.

When to Use Each?

1. **Bagging:**
 - Best for high-variance models like decision trees.
 - When you want to reduce overfitting.
2. **Boosting:**
 - Best for reducing bias.
 - Useful when your model underfits the data.
3. **AdaBoost:**
 - Best for simple models and small datasets.

- Avoid if data has significant noise or outliers.
4. **XGBoost:**
- Best for large and complex datasets.
 - Use when you need high performance and speed.

Classification Model Evaluation and Selection:

1. Sensitivity (True Positive Rate, Recall)

- **Definition:** Sensitivity measures how effectively the model identifies positive cases out of all actual positive cases.

- **Formula:**

$$\text{Sensitivity} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **Example:** In a cancer detection model:

- If there are 100 actual cancer cases and the model correctly identifies 90 of them, the sensitivity is:

$$\text{Sensitivity} = \frac{90}{90 + 10} = 0.9 \text{ (90\%)}$$

- **Importance:** High sensitivity ensures that few actual positive cases are missed (low false negatives).
- **Applications:** Medical diagnostics, fraud detection, and any scenario where missing a positive case is critical.

2. Specificity (True Negative Rate)

- **Definition:** Specificity measures how effectively the model identifies negative cases out of all actual negative cases.

- **Formula:**

$$\text{Specificity} = \frac{\text{True Negatives (TN)}}{\text{True Negatives (TN)} + \text{False Positives (FP)}}$$

- **Example:** In spam email filtering:

- If there are 200 legitimate emails and the model correctly identifies 180 as not spam, the specificity is:

$$\text{Specificity} = \frac{180}{180 + 20} = 0.9 \text{ (90\%)}$$

- **Importance:** High specificity ensures that legitimate negative cases are not wrongly classified as positive (low false positives).
- **Applications:** Spam filters, credit approval systems, and any scenario where false positives are costly.

3. Positive Predictive Value (PPV, Precision)

- **Definition:** PPV measures the probability that a positive prediction is correct.
- **Formula:**

$$PPV = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Example:** In fraud detection:
 - If the model predicts 50 transactions as fraudulent and 45 of them are truly fraudulent:

$$PPV = \frac{45}{45 + 5} = 0.9 \text{ (90\%)}$$

- **Importance:** High PPV means fewer false alarms, which reduces unnecessary interventions.
- **Applications:** Fraud detection, medical testing, and security systems.

4. Negative Predictive Value (NPV)

- **Definition:** NPV measures the probability that a negative prediction is correct.
- **Formula:**

$$NPV = \frac{\text{True Negatives (TN)}}{\text{True Negatives (TN)} + \text{False Negatives (FN)}}$$

- **Example:** In loan approval:
 - If the model rejects 100 applicants and 95 of them were truly unqualified:

$$NPV = \frac{95}{95 + 5} = 0.95 \text{ (95\%)}$$

- **Importance:** High NPV ensures that the model rarely misclassifies legitimate negative cases.
- **Applications:** Loan approvals, medical diagnosis, and quality control.

5. Lift Curves

- **Definition:** Lift curves show how much better the model performs compared to random guessing.
- **How to Interpret:**
 - The Y-axis represents the percentage of true positives identified.
 - The X-axis represents the percentage of the dataset used (ranked by predicted likelihood).
 - A steep initial curve indicates that the model effectively prioritizes the most likely positive cases.
- **Applications:** Marketing campaigns, where the goal is to target high-value customers efficiently.

6. Gain Curves

- **Definition:** Gain curves show the cumulative percentage of true positives captured as you increase the dataset size.
- **How to Interpret:**
 - The closer the curve is to the top-right corner, the better the model is at prioritizing positives.
 - The baseline (diagonal line) represents random guessing.

- **Applications:** Similar to lift curves, useful for evaluating ranking and targeting models.

7. ROC (Receiver Operating Characteristic) Curves

- **Definition:** The ROC curve illustrates the trade-off between sensitivity (true positive rate) and 1-specificity (false positive rate) at various threshold values.
- **Key Points:**
 - The curve closer to the top-left corner is better.
 - The Area Under the Curve (AUC) quantifies overall performance (closer to 1 is better).
- **Applications:** Comparing models to find the one with the best trade-off for the application, especially in imbalanced datasets.

8. Misclassification Cost Adjustment

- **Definition:** Adjusts the model to account for real-world costs associated with different types of errors.
- **Steps:**
 1. Assign costs to false positives and false negatives based on the application.
 2. Adjust the decision threshold or use cost-sensitive algorithms to minimize overall costs.
- **Example:**
 - In fraud detection, a false negative (missing a fraud) might cost \$1,000, while a false positive (flagging a legitimate transaction) costs \$10.
 - The model should focus on minimizing false negatives to reduce overall costs.

9. Decision Cost/Benefit Analysis

- **Definition:** Balances the benefits of correct predictions against the costs of incorrect predictions.
- **How to Perform:**
 1. Quantify the benefits of true positives and true negatives.
 2. Quantify the costs of false positives and false negatives.
 3. Calculate the net benefit or cost for different models.
- **Example:**
 - For a marketing campaign:
 - A true positive (targeting a responsive customer) results in a \$50 profit.
 - A false positive (targeting a non-responsive customer) costs \$5.
 - Analyze which model provides the best profit margin.